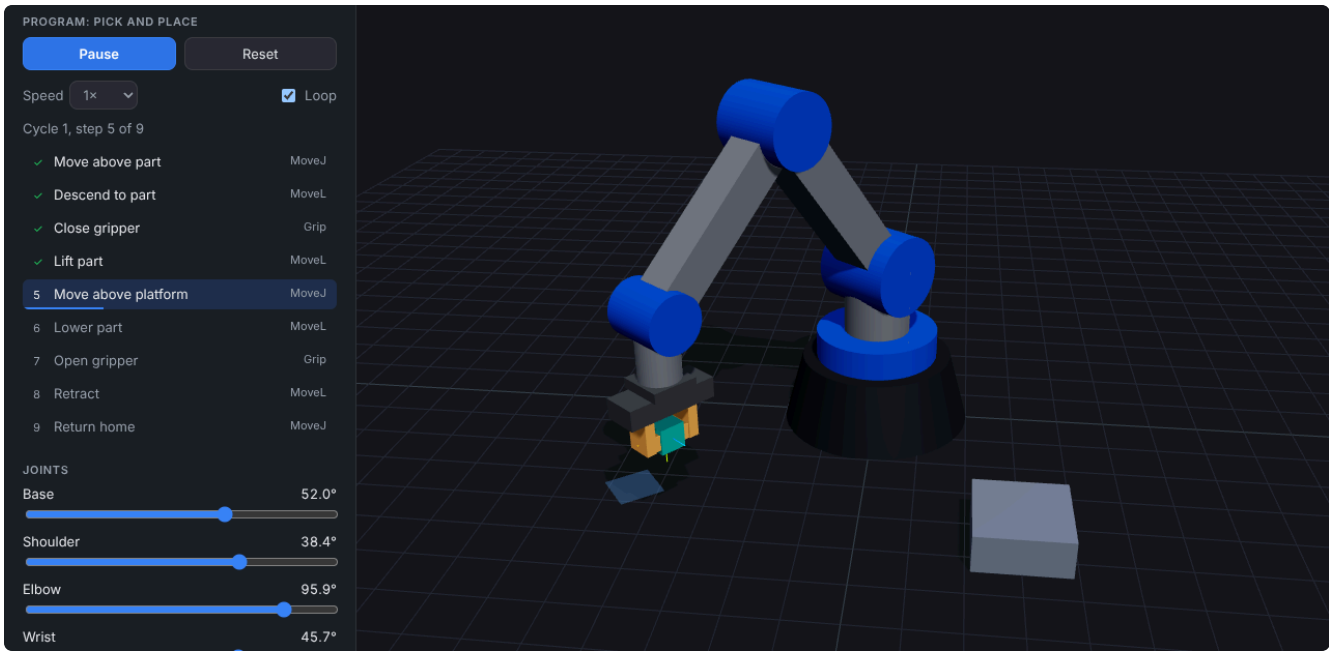
VENTION · PORTFOLIO WALKTHROUGH

Snack 2, Line by Line

A React control panel for the robot arm: inverse kinematics, a scripted pick-and-place program, and a 3D view that animates at full frame rate without re-rendering React, every file explained.



2 · The React Control Panel

Projects in `Portfolio/vention/` · Code quoted directly from the repository, with file and line numbers

Contents

How to read this book 3

Chapter 2 · The React Control Panel 4

2.1 Inverse kinematics 5

2.2 Joint space and Cartesian space 6

2.3 How React renders, and why the simulation lives outside it 6

2.4 useSyncExternalStore, and tearing 7

2.5 Effects, cleanup and StrictMode 7

2.6 Re-parenting with attach 8

Line by line 9

The robot program 9

Inverse kinematics 12

Running the program 16

The store between React and Three.js 24

The hook 31

Mounting the 3D view 31

The 3D side 33

Picking up the part 36

What changed in the stage 40

What changed in the gripper 42

The panel 43

Putting it together 51

Interview questions 52

Exercises 54

Appendix A · Running the projects 55

Appendix B · Glossary 56

BEFORE YOU START

How to read this book

This walkthrough covers Snack 2 on its own. It's also Chapter 2 of the combined book, *Five Snacks, Line by Line*, where every chapter has the same five parts:

1. **Concepts.** The ideas the code relies on, with diagrams, before any code.
2. **Line by line.** Every source file that matters, quoted from the repository in order, with each chunk explained. The code is sliced from the real files, so the line numbers match what you'll see in your editor.
3. **Going deeper.** Boxes inside the line-by-line parts that step back to what a production system, a full engine or a large team would do differently.
4. **Interview questions.** Questions this project invites, with model answers grounded in the code.
5. **Exercises.** Extensions to try, each with a hint.

Two reading paths work well. To prepare for a conversation about the project, read the concepts and the questions. To understand or extend the code, read the line-by-line parts with the project open next to the book.

The project

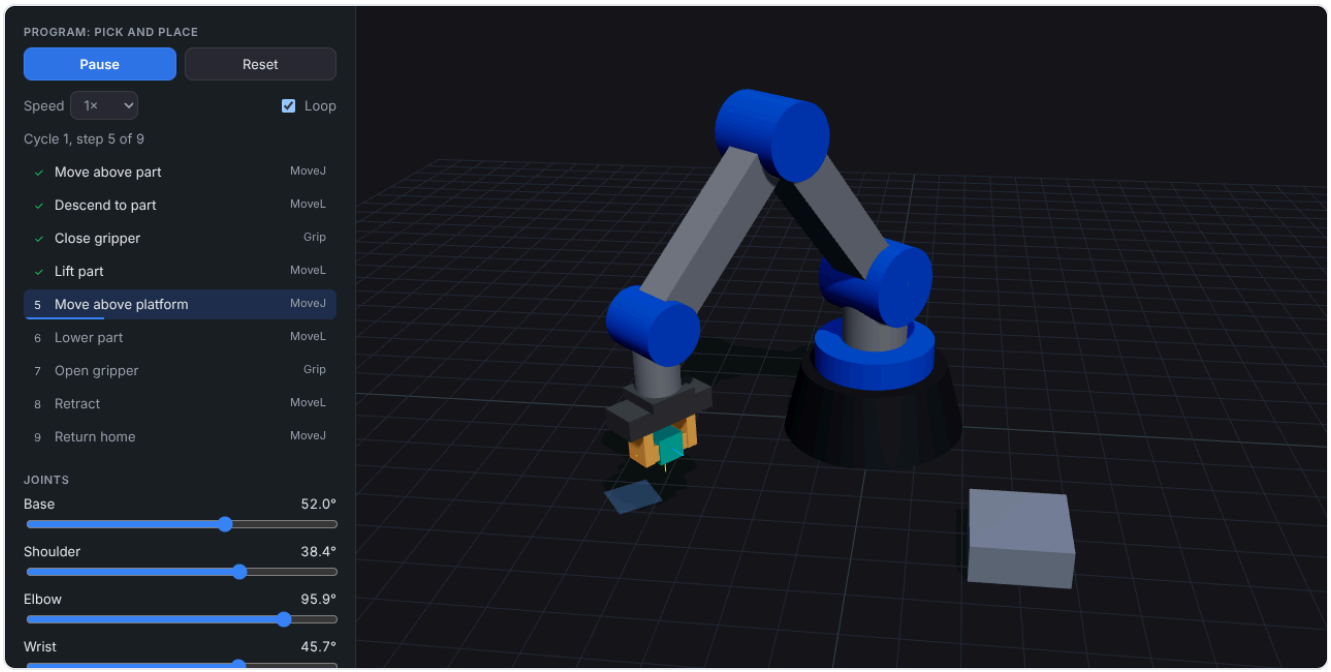
	Project	Stack	What it shows
2	**The React Control Panel**	React · Three.js · TypeScript	A React panel drives a pick-and-place program through inverse kinematics, while the 3D view animates at full frame rate without re-rendering React every frame.

CHAPTER 2 · `vention/snack2-control-panel`

The React Control Panel

A React panel drives a pick-and-place program through inverse kinematics, while the 3D view animates at full frame rate without re-rendering React every frame.

React · Three.js · TypeScript



Mid-run: the part has been picked off the floor and the arm is moving it to the platform (step 5 of 9).

The one idea to take away

The simulation lives outside React. Commands flow in from the panel; the 3D view reads the live state every frame:

`panel command` → `Simulation` → `SequencePlayer` → `sim.state` → `Viewer (every frame)`

React only gets an immutable snapshot, immediately when the step changes, otherwise at most 15 times a second:

`Simulation.publish()` → `snapshot` → `useSyncExternalStore` → `panel render (≤15×/s)`

CONCEPTS · 2.1

Inverse kinematics

Inverse kinematics (IK) runs FK backwards: given where the tool should be, find the joint angles. It's harder because the answer isn't unique. There may be **none** (out of reach), **a few** (elbow up or elbow down; base turned 180° with the shoulder reversed), or **infinitely many**, for arms with more joints than the task needs.

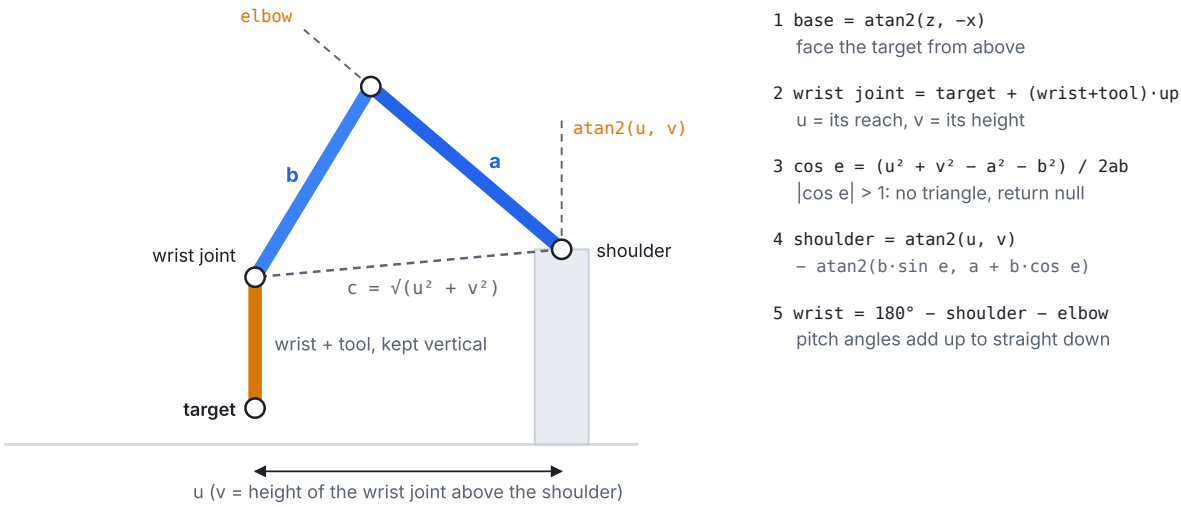


Figure 2.1. Closed-form inverse kinematics. Fixing the tool vertical turns the problem into a triangle with two known sides, solved by the law of cosines.

Solvers come in two families:

- **Analytical** (closed-form), like this one: exact, constant time, no iterations, but derived for one arm's geometry. Most industrial six-axis arms have a “spherical wrist” precisely so that a closed form exists; it yields up to eight solutions per pose.
- **Numerical**: start somewhere and iterate. Cyclic coordinate descent (CCD) turns each joint in turn to point the tool at the target; Jacobian methods (transpose, pseudo-inverse, damped least squares) step along the direction that reduces the error; FABRIK works on joint positions directly. They handle any arm but can be slow, jittery or stuck in local minima.

The trick that makes Snack 2's solver closed-form is **fixing the orientation**: the tool must point straight down. That uses up the fourth joint, leaving a two-link problem in a plane, solved by the law of cosines. Joint limits are checked afterwards, because geometry alone doesn't know about them.

CONCEPTS · 2.2

Joint space and Cartesian space

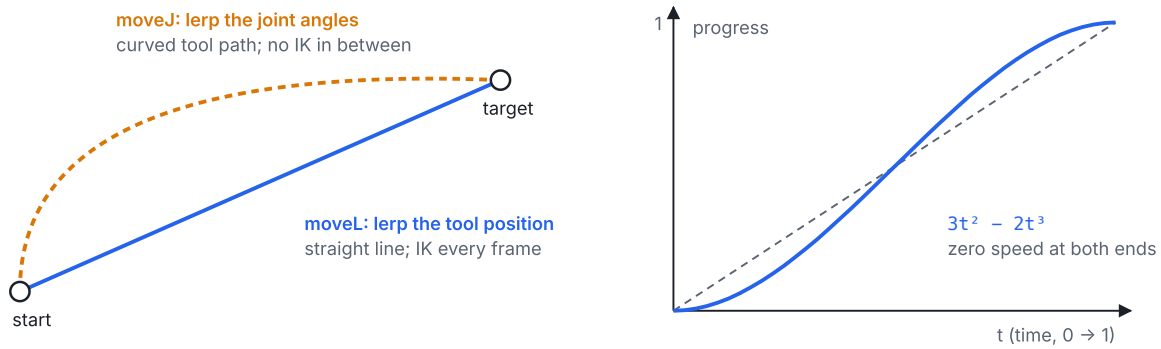


Figure 2.2. Left: the two robot moves between the same two points. Right: smoothstep easing, used for both, so every move starts and stops gently.

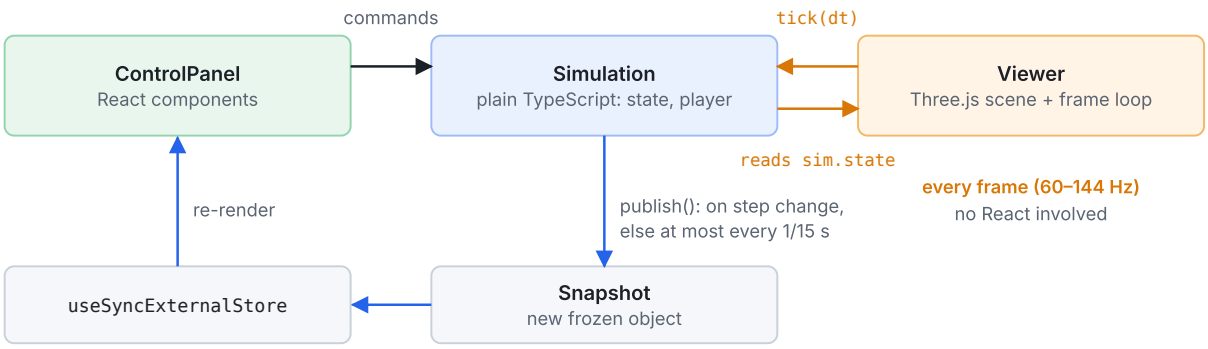
- **moveJ** interpolates the joint angles. All joints start and stop together, it's the fastest way between two poses, and it can never pass through a singularity. The tool's path is a curve that's hard to predict, so it's for open space.
- **moveL** interpolates the tool position and solves IK at every step. It's needed for approach and retreat, where a curved path would knock the part over. It can fail halfway (a point on the line unreachable, or a singularity), so real controllers validate the whole path before moving.

Both use **smoothstep** easing, $3t^2 - 2t^3$, whose speed $6t(1 - t)$ is zero at both ends, so moves don't start or stop with a jerk in velocity. Acceleration still jumps at the ends; real controllers use trapezoidal or S-curve velocity profiles limited by each joint's maximum speed and acceleration, and size each move by its slowest joint.

CONCEPTS · 2.3

How React renders, and why the simulation lives outside it

A React **render** is a call to your component functions, which return a description of the UI. React compares it with the previous description (**reconciliation**) and applies the differences to the DOM (the **commit**). A render happens when state, props or context change. Each one is cheap, but it runs on the same main thread as the 3D frame loop, inside the same 16.7 ms budget at 60 fps.



Fast path (orange): the 3D view reads mutable state directly. Slow path (blue): React sees an immutable snapshot, throttled.

Figure 2.3. Snack 2's data flow. The simulation lives outside React, so animation never waits for a React render.

So Snack 2 keeps the fast-changing state in a plain TypeScript class. The render loop mutates and reads it directly, 60 or more times a second. React receives an **immutable snapshot**: immediately on discrete events (step changed, paused), and at most 15 times a second for continuous values like joint angles. That's the same pattern state libraries such as Zustand use, and the advice react-three-fiber gives: mutate in the frame loop, don't call `setState` per frame.

CONCEPTS · 2.4

useSyncExternalStore, and tearing

With concurrent rendering, React can pause a render partway and continue later. If components read a mutable store directly while rendering, two of them could see different values in the same commit: **tearing**. `useSyncExternalStore` prevents it. React reads the store only through `getSnapshot`, checks it again before committing, and re-renders synchronously if it changed.

- `subscribe(listener)` registers a callback and returns a function that unregisters it.
- `getSnapshot()` must return the **same object** (by `Object.is`) until the data changes. Returning a fresh object each call makes React think the data changed every time, and it loops until it gives up with an error.

CONCEPTS · 2.5

Effects, cleanup and StrictMode

`useEffect` runs after React commits, and its cleanup function runs before the effect runs again and when the component unmounts. In development, StrictMode deliberately mounts, unmounts and remounts every component once, so a missing cleanup shows up immediately instead of months later.

For a Three.js view, cleanup means: stop the animation loop, remove event listeners, dispose geometries, materials, textures and controls, and dispose the renderer. Browsers cap the number of live WebGL contexts (Chrome allows around 16) and silently drop the oldest, so leaks become broken canvases. Async work started before unmount, like loading the gripper, must check a `disposed` flag when it finishes.

CONCEPTS · 2.6

Re-parenting with attach

Picking up the part is a change of parent. `parent.add(child)` keeps the child's **local** transform, so its world position jumps to wherever that offset lands under the new parent. `parent.attach(child)` keeps its **world** transform: it computes the new local matrix as `inverse(parent.matrixWorld) × child.matrixWorld`. Grasp is `gripper.attach(part)`, release is `scene.attach(part)`, and the part never moves at the moment of either.

CHAPTER 2

Line by line

12 parts, in the order data flows through the code.

PART 2.1 · `src/sim/program.ts`

The robot program

What the robot should do, written as data. No logic runs here; it's the script the player follows.

The step types

src/sim/program.ts · lines 3–18

```
/**
 * The robot program: a list of steps, in the style of an industrial robot's
 * teach pendant or Vention's MachineLogic.
 *
 * - moveJ:   joint move. Solve IK for the end point once, then interpolate each
 *            joint. Fast and smooth, but the tool's path in space is a curve.
 * - moveL:   linear move. Interpolate the tool position in a straight line and
 *            solve IK every frame. Used for approaching and leaving the part.
 * - gripper: open or close the fingers to a gap (meters).
 * - home:    joint move back to the home pose.
 */
export type Step =
  | { kind: "moveJ"; label: string; to: Vec3; duration: number }
  | { kind: "moveL"; label: string; to: Vec3; duration: number }
  | { kind: "gripper"; label: string; gap: number; duration: number }
  | { kind: "home"; label: string; duration: number };
```

A **discriminated union**: every step has a `kind` field, and TypeScript uses it to know which other fields exist. A `moveJ` step has a `to` point; a `gripper` step has a `gap` instead. When the player does `switch (step.kind)`, TypeScript narrows the type in each branch, so `step.to` is only allowed where it actually exists.

All four kinds share `label` (shown in the panel) and `duration` (seconds at 1× speed).

The doc comment explains the difference that matters most: **moveJ** interpolates the joint angles, so the tool follows a curve; **moveL** interpolates the tool position, so the tool travels in a straight line. Real robot languages have the same two moves with the same names.

The work-cell layout

src/sim/program.ts · lines 20–30

```
// ---- Work cell layout (meters) ----

export const PART_SIZE = 0.05; // cube edge
export const GAP_OPEN = 0.096; // distance between the grip pads
export const GAP_CLOSED = 0.006;

export const PLATFORM = { x: 0.3, z: 0.45, size: 0.18, height: 0.075 };

/** Where the part's center sits when resting at each station. */
export const FLOOR_SPOT: Vec3 = { x: -0.45, y: PART_SIZE / 2, z: 0.35 };
export const PLATFORM_SPOT: Vec3 = { x: PLATFORM.x, y: PLATFORM.height + PART_SIZE / 2, z:
PLATFORM.z };
```

- `PART_SIZE`: the cube is 5 cm on each side.
- `GAP_OPEN` / `GAP_CLOSED`: how far apart the grip pads are when fully open (9.6 cm) or fully closed (6 mm). These match the fingers' travel in the Snack 1 gripper model.
- `PLATFORM`: an 18 cm square block, 7.5 cm tall, centered at x = 0.3, z = 0.45.
- `FLOOR_SPOT` and `PLATFORM_SPOT`: where the **center** of the part sits when it's resting at each station. That's half the part's height above the surface, which is why both add `PART_SIZE / 2`.

Positions are the tool center point targets too: the TCP sits between the fingertips, so putting it on the part's center means the fingers close around the middle of the cube.

Helpers

src/sim/program.ts · lines 32–35

```
const CLEARANCE = 0.15; // approach height above a station
const RELEASE_DROP = 0.002; // let go 2 mm above the surface so the part settles

const above = (p: Vec3, height = CLEARANCE): Vec3 => ({ ...p, y: p.y + height });
```

`CLEARANCE` is how high above a station the arm approaches from (15 cm). `RELEASE_DROP` makes the robot let go 2 mm above the surface, so the part settles under gravity instead of being pushed into the platform.

`above()` returns a copy of a point raised by some height. `{ ...p, y: ... }` copies `x` and `z` and replaces `y`, without changing the original point. The default argument means `above(p)` uses `CLEARANCE`.

One pick-and-place cycle

src/sim/program.ts · lines 37–51

```
/** One pick-and-place cycle, ending back at home. */
export function pickAndPlace(from: Vec3, to: Vec3, fromName: string, toName: string): Step[] {
  const release = above(to, RELEASE_DROP);
  return [
    { kind: "moveJ", label: `Move above ${fromName}`, to: above(from), duration: 2 },
    { kind: "moveL", label: "Descend to part", to: from, duration: 1.2 },
    { kind: "gripper", label: "Close gripper", gap: PART_SIZE, duration: 0.6 },
    { kind: "moveL", label: "Lift part", to: above(from), duration: 1 },
    { kind: "moveJ", label: `Move above ${toName}`, to: above(release), duration: 2.2 },
    { kind: "moveL", label: "Lower part", to: release, duration: 1.2 },
    { kind: "gripper", label: "Open gripper", gap: GAP_OPEN, duration: 0.5 },
    { kind: "moveL", label: "Retract", to: above(release), duration: 0.9 },
    { kind: "home", label: "Return home", duration: 2 },
  ];
}
```

This builds the nine steps of one cycle from two stations. Reading it top to bottom is reading the robot's job:

1. **moveJ** to 15 cm above the part: the big, fast move, where the exact path doesn't matter.
2. **moveL** straight down onto the part. A straight line matters here: a curved path could knock the part over.
3. Close the gripper to exactly `PART_SIZE`, so the pads stop on the part's faces.
4. **moveL** straight back up.
5. **moveJ** across to above the drop-off point.
6. **moveL** down to the release height.
7. Open the gripper. The part is released and drops 2 mm.
8. **moveL** straight up, clear of the part.
9. Return to the home pose.

Template strings like `` `Move above ${fromName}` `` give each step a readable label for the panel.

Looping without running out of work

src/sim/program.ts · lines 53–58

```
/** Even cycles move the part floor → platform, odd cycles bring it back, so looping never runs
out of work. */
export function programForCycle(cycle: number): Step[] {
  return cycle % 2 === 0
    ? pickAndPlace(FLOOR_SPOT, PLATFORM_SPOT, "part", "platform")
    : pickAndPlace(PLATFORM_SPOT, FLOOR_SPOT, "part", "floor spot");
}
```

If the program always went floor → platform, a second loop would try to pick from an empty floor. Instead, even cycles (0, 2, 4…) carry the part to the platform and odd cycles carry it back, so **Loop** can run forever. `cycle % 2 === 0` is the remainder after dividing by 2: 0 for even numbers.

The player calls this function to get each cycle's steps, so the program can depend on the cycle number without the player knowing anything about stations.

PART 2.2 · `src/arm/ik.ts`

Inverse kinematics

Forward kinematics (Snack 1) answers “given these joint angles, where is the tool?”. Inverse kinematics answers the reverse: “I want the tool here, what angles do I need?”. It's what lets the program be written as points in space.

The signature and the idea

src/arm/ik.ts · lines 4–16

```
/**
 * Inverse kinematics for a top-down grasp: find the joint angles that put the
 * tool center point on `target` with the gripper pointing straight down.
 *
 * Returns [base, shoulder, elbow, wrist] in degrees, or null if out of reach.
 *
 * Fixing the tool orientation makes this solvable in closed form:
 *  1. The base yaw is just the compass direction to the target.
 *  2. With the tool vertical, the wrist joint sits straight above the target,
 *     so shoulder + elbow form a two-link arm reaching for that point.
 *  3. The wrist takes whatever angle makes the three pitch angles sum to 180°.
 */
export function solveTopDown(d: ArmDimensions, target: Vec3): number[] | null {
```

General IK has many solutions or none, and usually needs an iterative solver. This one is solvable directly because it **fixes the tool orientation**: pointing straight down, which is what you want for picking things up from above.

It returns degrees (matching the rest of the app) or `null` when the point is out of reach, so callers have to handle failure.

Step 1: base yaw

src/arm/ik.ts · lines 17–20

```
// 1. Base yaw. The arm reaches along its local -X, so FK gives
//    x = -reach·cos(base), z = reach·sin(base).
const reach = Math.hypot(target.x, target.z);
const base = Math.atan2(target.z, -target.x);
```

Seen from above, the base just needs to turn to face the target. `Math.hypot(x, z)` is the horizontal distance from the base axis ($\sqrt{x^2 + z^2}$).

`Math.atan2(y, x)` gives the angle of a direction, handling all four quadrants. The arguments are `(z, -x)` rather than `(z, x)` because of how the arm was built in Snack 1: it reaches along its local **−X**, and Snack 1's forward kinematics gives `x = -reach·cos(base)` and `z = reach·sin(base)`. Solving those two for `base` gives this line.

Step 2: reduce to a two-link arm

src/arm/ik.ts · lines 22–27

```
// 2. Two-link problem in the arm's vertical plane, measured from the shoulder joint:
//    u = horizontal distance, v = height of the wrist joint above the shoulder.
const u = reach;
const v = target.y + d.wrist + d.tool - (d.baseHeight + d.shoulderHeight);
const a = d.upperArm;
const b = d.forearm;
```

With the gripper pointing straight down, the wrist joint must be directly above the target, higher by the length of the wrist link plus the tool (`d.wrist + d.tool`). So instead of solving for the tool, solve for the wrist joint.

That leaves only the upper arm (`a`) and forearm (`b`) to place, in the arm's vertical plane. `u` is the horizontal distance to the wrist joint and `v` is its height above the shoulder joint (the shoulder sits at `baseHeight + shoulderHeight`). This is the classic two-link planar arm problem.

The elbow, by the law of cosines

src/arm/ik.ts · lines 29–32

```
// Law of cosines gives the elbow bend. Outside [-1, 1] means the point is too far or too
close.
const cosElbow = (u * u + v * v - a * a - b * b) / (2 * a * b);
if (cosElbow < -1 || cosElbow > 1) return null;
const elbow = Math.acos(cosElbow); // 0..180°, the "elbow up" solution
```

The upper arm, forearm and the straight line from shoulder to wrist form a triangle with sides `a`, `b` and $\sqrt{(u^2 + v^2)}$. The law of cosines relates the elbow bend to those sides, and this line solves it for the cosine of the bend.

A cosine must be between −1 and 1. If it isn't, no triangle exists: the target is farther than the arm can stretch, or closer than it can fold. That's the `return null`.

`Math.acos` returns 0 to 180°. Taking the positive answer picks the “elbow up” pose; the negative angle would be the mirror-image “elbow down” solution, which reaches the same point.

The shoulder

src/arm/ik.ts · lines 34–36

```
// Shoulder angle = direction to the wrist joint, minus the angle the elbow bend adds.
// Angles here are measured from vertical, matching how the joints are built.
const shoulder = Math.atan2(u, v) - Math.atan2(b * Math.sin(elbow), a + b * Math.cos(elbow));
```

Two angles, subtracted:

- `Math.atan2(u, v)`: the direction from the shoulder to the wrist joint, measured from vertical (the arm's zero pose points straight up, so angles are measured from vertical throughout).
- `Math.atan2(b·sin(elbow), a + b·cos(elbow))`: how far the bent elbow swings the wrist away from the upper arm's own direction.

Aim at the target, then back off by the amount the elbow bend adds, and the upper arm is at the right angle.

Step 3: the wrist keeps the tool vertical

src/arm/ik.ts · lines 38–42

```
  // 3. Keep the tool vertical: shoulder + elbow + wrist = 180°.
  const wrist = Math.PI - shoulder - elbow;

  return [base, shoulder, elbow, wrist].map((angle) => angle / DEG);
}
```

The three pitch joints are parallel, so their angles simply add up. The tool points straight down when the total is 180° (π radians), so the wrist takes whatever is left over.

The angles were computed in radians, since that's what `Math` uses. The last line converts all four to degrees: `DEG` is π/180, so dividing by it converts radians to degrees.

Joint limits

src/arm/ik.ts · lines 44–46

```
export function withinLimits(specs: readonly JointSpec[], anglesDeg: readonly number[]):
boolean {
  return specs.every((s, i) => anglesDeg[i] >= s.min && anglesDeg[i] <= s.max);
}
```

IK only knows geometry. It can return an elbow angle of 150° even though the joint stops at 140°. `every` returns true only if each angle is within its joint's min and max, and the player checks this before using any IK result.

GOING DEEPER

A numerical solver in a paragraph

Cyclic coordinate descent, for comparison: for each joint from the wrist back to the base, rotate that joint so the line from it to the tool points at the target (clamped to its limits); repeat the sweep until the tool is within a millimeter or a maximum number of sweeps is reached. It works for any chain and any number of joints, but tends to curl the last joints first, gives no control over which solution you get, and can take many sweeps. The closed form is exact in one pass, which is why it's preferred whenever the geometry allows one.

PART 2.3 · `src/sim/SequencePlayer.ts`

Running the program

The player steps through the program, turning each step into joint angles and a gripper gap, frame by frame. It imports no Three.js or React, so the tests can run the whole program in plain Node.

The state it writes

src/sim/SequencePlayer.ts · lines 6–12

```
/** The commanded state of the robot. Joints in degrees, gripper gap in meters. */
export interface ArmState {
  joints: number[];
  gap: number;
}

export type PlayerStatus = "idle" | "playing" | "paused" | "done" | "error";
```

`ArmState` is the **commanded** robot: four joint angles in degrees and the gap between the grip pads in meters. The player writes into it; the 3D viewer reads from it. It's a plain object, deliberately not a Three.js object.

`PlayerStatus` is a union of string literals, so a typo like `"plaing"` is a compile error.

Easing and interpolation

src/sim/SequencePlayer.ts · lines 14–17

```
/** Smoothstep: starts and ends with zero velocity, so moves don't jerk. */
const ease = (t: number) => t * t * (3 - 2 * t);
const lerp = (a: number, b: number, t: number) => a + (b - a) * t;
const format = (p: Vec3) => `(${p.x.toFixed(2)}, ${p.y.toFixed(2)}, ${p.z.toFixed(2)})`;
```

- `ease` is **smoothstep**: for t from 0 to 1 it also goes from 0 to 1, but starts and ends slowly. Real robots accelerate and decelerate rather than jumping to full speed, and this gives the same look.
- `lerp` (linear interpolation) returns the value a fraction `t` of the way from `a` to `b`. At t = 0 it's `a`, at t = 1 it's `b`.
- `format` turns a point into readable text for error messages.

Fields

src/sim/SequencePlayer.ts · lines 23–41

```
export class SequencePlayer {
  status: PlayerStatus = "idle";
  steps: Step[];
  index = 0;
  elapsed = 0; // seconds into the current step (scaled by speed)
  cycle = 0;
  loop = true;
  speed = 1;
  error: string | null = null;

  private readonly dims: ArmDimensions;
  private readonly specs: readonly JointSpec[];
  private readonly program: (cycle: number) => Step[];

  // Captured when a step begins, so the step interpolates from where the arm really is.
  private startJoints: number[] = [];
  private startGap = 0;
  private startTcp: Vec3 = { x: 0, y: 0, z: 0 };
  private targetJoints: number[] | null = null;
```

The public fields are what the panel shows: status, the current cycle's steps, which step it's on, how far into it (`elapsed`), the cycle number, and the loop, speed and error settings.

The `start…` fields are captured **when each step begins**: where the joints, gripper and tool were at that moment. Every step interpolates from where the arm actually is, never from where the program assumed it would be. `targetJoints` holds the IK answer for joint moves.

Constructor and progress

src/sim/SequencePlayer.ts · lines 43–54

```
constructor(dims: ArmDimensions, specs: readonly JointSpec[], program: (cycle: number) =>
Step[]) {
  this.dims = dims;
  this.specs = specs;
  this.program = program;
  this.steps = program(0);
}

/** 0..1 through the current step. */
get progress(): number {
  const step = this.steps[this.index];
  return step ? Math.min(1, this.elapsed / step.duration) : 0;
}
```

The player is given the arm's dimensions and joint limits (for IK and FK) and a **function** that returns the steps for a cycle. Passing `programForCycle` in, rather than importing it, keeps the player reusable: a test could give it any program.

`get progress()` is a getter: read like a property (`player.progress`) but computed each time. It's how far through the current step the player is, from 0 to 1, for the panel's progress bar.

Play, pause and stop

src/sim/SequencePlayer.ts · lines 56–77

```
/**
 * Idle or error: start from cycle 0. Done: start the next cycle. Paused: resume.
 * In every case the current step restarts from the arm's current pose, so if
 * someone moved a joint while paused, the arm glides back instead of jumping.
 */
play(state: ArmState): void {
  if (this.status === "playing") return;
  if (this.status === "idle" || this.status === "error") this.rewind();
  this.error = null;
  this.status = "playing";
  this.beginStep(state);
}

pause(): void {
  if (this.status === "playing") this.status = "paused";
}

stop(): void {
  this.rewind();
  this.status = "idle";
  this.error = null;
}
```

`play` handles three situations:

- From **idle** or **error**, it rewinds to cycle 0 and starts fresh.
- From **done** (one cycle finished, loop off), it leaves the cycle counter alone, so the next play runs the return trip.
- From **paused**, it resumes the same step.

All three end with `beginStep(state)`, which re-captures the start pose. So if you dragged a slider while paused, the step re-plans from your new pose and the arm glides back on course instead of jumping.

`pause` only changes the status, so the arm freezes where it is. `stop` rewinds and goes idle.

Advancing one frame

src/sim/SequencePlayer.ts · lines 79–85

```
/** Advances by dt seconds, writing the new joint angles and gap into `state`. */
update(dt: number, state: ArmState): void {
  if (this.status !== "playing") return;
  const step = this.steps[this.index];
  this.elapsed += dt * this.speed;
  const t = Math.min(1, this.elapsed / step.duration);
  const s = ease(t);
```

The 3D viewer calls this every frame with `dt`, the seconds since the last frame. Multiplying by `speed` makes 2× speed advance the program twice as far per frame. `t` is the fraction of the step completed (capped at 1), and `s` is that fraction eased.

Joint moves

src/sim/SequencePlayer.ts · lines 87–95

```
switch (step.kind) {
  case "moveJ":
  case "home": {
    const target = this.targetJoints!;
    for (let i = 0; i < state.joints.length; i++) {
      state.joints[i] = lerp(this.startJoints[i], target[i], s);
    }
    break;
  }
```

For `moveJ` and `home`, each joint goes from its start angle to its target angle, all with the same eased `s`, so all joints start and finish together.

The `!` after `this.targetJoints` tells TypeScript “this isn't null here”. That's safe because `beginStep` always sets it for these two kinds, or puts the player in the error state so `update` never gets here.

The loop writes into the existing `state.joints` array instead of creating a new one: no allocation every frame.

Linear moves

src/sim/SequencePlayer.ts · lines 96–109

```
case "moveL": {
  const p = {
    x: lerp(this.startTcp.x, step.to.x, s),
    y: lerp(this.startTcp.y, step.to.y, s),
    z: lerp(this.startTcp.z, step.to.z, s),
  };
  const q = solveTopDown(this.dims, p);
  if (!q || !withinLimits(this.specs, q)) {
    this.fail(`Can't reach ${format(p)} during "${step.label}"`);
    return;
  }
  for (let i = 0; i < q.length; i++) state.joints[i] = q[i];
  break;
}
```

For `moveL`, the player interpolates the **tool position** (x, y and z each lerped from the start to the target), then asks IK for the joint angles that put the tool there. Doing this every frame is what makes the path a straight line.

If a point along the line is unreachable or breaks a joint limit, the player stops with a readable error rather than moving the arm into an impossible pose.

Gripper steps

src/sim/SequencePlayer.ts · lines 110–113

```
  case "gripper":
    state.gap = lerp(this.startGap, step.gap, s);
    break;
}
```

The gap goes from where it was to the step's target. The joints don't change during a gripper step.

Finishing a step

src/sim/SequencePlayer.ts · lines 115–130

```
  if (t < 1) return;

  // Step finished: move to the next one, or the next cycle.
  this.index++;
  if (this.index >= this.steps.length) {
    this.cycle++;
    this.steps = this.program(this.cycle);
    this.index = 0;
    if (!this.loop) {
      this.status = "done";
      this.elapsed = 0;
      return;
    }
  }
  this.beginStep(state);
}
```

Until `t` reaches 1, there's nothing more to do this frame. Once it does, move to the next step. Past the last step, the cycle is over: bump the cycle number and ask for the next cycle's steps (which, from program.ts, go the other direction).

With loop off, stop here with status **done**. Otherwise, or if there are more steps, `beginStep` sets up the next one immediately, so there's no idle frame between steps.

Setting up a step

src/sim/SequencePlayer.ts · lines 132–157

```
private rewind(): void {
  this.cycle = 0;
  this.steps = this.program(0);
  this.index = 0;
  this.elapsed = 0;
}

private beginStep(state: ArmState): void {
  const step = this.steps[this.index];
  this.elapsed = 0;
  this.startJoints = [...state.joints];
  this.startGap = state.gap;
```

```
  this.startTcp = forwardKinematics(this.dims, state.joints.map((a) => a * DEG));
  this.targetJoints = null;

  if (step.kind === "moveJ") {
    const q = solveTopDown(this.dims, step.to);
    if (!q || !withinLimits(this.specs, q)) {
      this.fail(`Can't reach ${format(step.to)} for "${step.label}"`);
      return;
    }
    this.targetJoints = q;
  } else if (step.kind === "home") {
    this.targetJoints = this.specs.map((s) => s.home);
  }
}
```

`rewind` returns to the start of cycle 0. `beginStep` snapshots the starting point: a copy of the joint angles (`[...state.joints]` makes a new array so later changes don't affect it), the gap, and the tool position from forward kinematics (which takes radians, hence `a * DEG`).

For `moveJ`, IK runs **once**, here, for the end point; the frames in between only interpolate. For `home`, the target is just each joint's home angle. Either failure becomes an error.

Errors

src/sim/SequencePlayer.ts · lines 159–162

```
private fail(message: string): void {
  this.status = "error";
  this.error = message;
}
```

One place to enter the error state, so the status and message always change together.

GOING DEEPER

Time-based steps make the robot testable

The player never looks at the clock. It's given `dt`, advances elapsed time, and computes progress as elapsed ÷ duration. Pause is simply “ignore dt”, and speed multiplies it. Because nothing depends on rendering, a test can drive the player in plain Node with a fixed dt and check things a person can't easily see: that the gripper closes exactly on the part, and that over two looping cycles it never touches the floor or the platform.

PART 2.4 · `src/sim/Simulation.ts`

The store between React and Three.js

This is the heart of Snack 2's design. The simulation lives outside React. The 3D view reads it every frame; React gets an occasional snapshot.

What React sees

src/sim/Simulation.ts · lines 6–19

```
/** What React sees: an immutable copy of the simulation, replaced (never mutated) on each
publish. */
export interface Snapshot {
  joints: readonly number[];
  gap: number;
  tcp: Vec3;
  status: PlayerStatus;
  steps: readonly Step[];
  stepIndex: number;
  stepProgress: number;
  cycle: number;
  speed: number;
  loop: boolean;
  error: string | null;
}
```

A `Snapshot` is a frozen copy of everything the panel displays. `readonly` stops components from modifying it. Each publish creates a **new** snapshot object rather than changing the old one, which is how React knows something changed: it compares the object references.

Tuning constants

src/sim/Simulation.ts · lines 21–22

```
const PUBLISH_INTERVAL = 1 / 15; // seconds: React re-renders at most ~15x/s while things move
const GRIPPER_SPEED = 0.25; // m/s for manual open/close
```

`PUBLISH_INTERVAL` caps how often React hears about continuous motion: 1/15 s, so at most about 15 panel renders a second while the arm moves, however fast the screen refreshes. `GRIPPER_SPEED` is for the manual Open/Close buttons.

The live state

src/sim/Simulation.ts · lines 24–49

```
/**
 * The single source of truth for the robot, living outside React.
 *
 * The 3D viewer calls tick() every frame (60+ times a second) and reads `state`
 * directly. React reads `snapshot` through useSyncExternalStore, and only gets a
 * new one when something changes, throttled while the arm is moving. So the
 * scene animates at full frame rate while the panel re-renders a few times a second.
 */
export class Simulation {
  /** Live, mutable state. The viewer reads this every frame. */
  readonly state: ArmState = { joints: JOINTS.map((j) => j.home), gap: GAP_OPEN };
  readonly player = new SequencePlayer(ARM, JOINTS, programForCycle);
  /** Bumped on reset, so the viewer knows to put the part back. */
  resetVersion = 0;
  /** Frames simulated so far, shown in the panel's stats line. */
  frames = 0;

  private manualGap = GAP_OPEN;
  private snapshot: Snapshot;
  private readonly listeners = new Set<() => void>();
  private sincePublish = 0;
  private wasMoving = false;

  constructor() {
    this.snapshot = this.makeSnapshot();
  }
```

`state` is the live, mutable robot that the player writes and the viewer reads, 60 or more times a second. React never reads it directly.

`resetVersion` is a counter instead of an event. Reset increments it; the viewer remembers the last value it saw, and when they differ it puts the part back. No callbacks to register or clean up.

`manualGap` is where the manual gripper buttons want the gap to be. `listeners` holds React's subscription callbacks. `sincePublish` and `wasMoving` drive the throttling in `tick`.

The external-store contract

src/sim/Simulation.ts · lines 51–59

```
// ---- The external-store contract for useSyncExternalStore ----
// Arrow functions so `this` is bound and React gets the same function every time.

subscribe = (listener: () => void): (() => void) => {
  this.listeners.add(listener);
  return () => this.listeners.delete(listener);
};

getSnapshot = (): Snapshot => this.snapshot;
```

React's `useSyncExternalStore` hook needs exactly two functions from a store:

- `subscribe(listener)`: remember this callback, call it whenever the data changes, and return a function that unsubscribes.
- `getSnapshot()`: return the current data. It must return the **same object** until something changes; otherwise React would re-render forever.

They're written as arrow-function properties rather than methods. An arrow function captures `this`, so React can call `sim.subscribe` on its own without losing which simulation it belongs to. And because they're created once, React receives the same function on every render and doesn't resubscribe.

Commands from the panel

src/sim/Simulation.ts · lines 61–103

```
// ---- Commands from the UI ----

play(): void {
  this.player.play(this.state);
  this.publish();
}

pause(): void {
  this.interrupt();
  this.publish();
}

reset(): void {
  this.player.stop();
```

```
  JOINTS.forEach((j, i) => (this.state.joints[i] = j.home));
  this.state.gap = this.manualGap = GAP_OPEN;
  this.resetVersion++;
  this.publish();
}

/** Manual jog. Takes control away from the program, like a robot's teach pendant. */
setJoint(index: number, degrees: number): void {
  this.interrupt();
  const spec = JOINTS[index];
  this.state.joints[index] = clamp(degrees, spec.min, spec.max);
  this.publish();
}

setGripper(open: boolean): void {
  this.interrupt();
  this.manualGap = open ? GAP_OPEN : GAP_CLOSED;
  this.publish();
}

setSpeed(speed: number): void {
  this.player.speed = speed;
  this.publish();
}

setLoop(loop: boolean): void {
  this.player.loop = loop;
  this.publish();
}
```

Every button and slider in the panel calls one of these. Each changes the state, then calls `publish()` so the panel updates immediately.

- `play` / `pause` pass through to the player.
- `reset` stops the program, puts the joints and gripper back to home, and bumps `resetVersion` for the part.
- `setJoint` is a manual jog. It first **interrupts** the program, the way grabbing a real robot's teach pendant takes control away from the running program, then sets the angle clamped to the joint's limits.
- `setGripper` also interrupts, then sets the manual gap target. The gap itself moves gradually in `tick`.

- `setSpeed` / `setLoop` change player settings.

tick: once per frame

src/sim/Simulation.ts · lines 105–122

```
// ---- Called by the viewer once per frame ----

tick(dt: number): void {
  this.frames++;
  const { status, index } = this.player;
  let moving = false;

  if (this.player.status === "playing") {
    this.player.update(dt, this.state);
    moving = true;
  } else if (this.state.gap !== this.manualGap) {
    const step = GRIPPER_SPEED * dt;
    this.state.gap =
      this.state.gap < this.manualGap
        ? Math.min(this.manualGap, this.state.gap + step)
        : Math.max(this.manualGap, this.state.gap - step);
    moving = true;
  }
```

The viewer calls this every frame. It notes the status and step before doing anything, so it can tell afterwards whether they changed.

If the program is playing, the player advances. Otherwise, if the manual gripper target differs from the current gap, the gap moves toward it at `GRIPPER_SPEED`, without overshooting (`Math.min` / `Math.max`). Either way, `moving` records that something changed this frame.

Deciding when to tell React

src/sim/Simulation.ts · lines 124–131

```
  // Publish right away when the step or status changes, otherwise at most
  // every PUBLISH_INTERVAL while moving, plus once more when motion stops.
  this.sincePublish += dt;
  const discreteChange = status !== this.player.status || index !== this.player.index;
  const stopped = this.wasMoving && !moving;
  if (discreteChange || stopped || (moving && this.sincePublish >= PUBLISH_INTERVAL))
this.publish();
  this.wasMoving = moving;
}
```

This is the throttle, and the reason the panel doesn't re-render 60 times a second. It publishes when:

- a **discrete** change happened (a new step started, or the status changed), immediately, so the step list never lags;
- motion just **stopped**, once more, so the final values shown are exact;
- something is **moving** and at least `PUBLISH_INTERVAL` has passed since the last publish.

When nothing moves, nothing publishes: an idle app causes zero React renders.

Internals

src/sim/Simulation.ts · lines 133–164

```
// ---- Internals ----

/** If the program is running, pause it and hold the gripper where it is. */
private interrupt(): void {
  if (this.player.status === "playing") {
    this.player.pause();
    this.manualGap = this.state.gap;
  }
}

private publish(): void {
  this.snapshot = this.makeSnapshot();
  this.sincePublish = 0;
  this.listeners.forEach((listener) => listener());
}
```

```
private makeSnapshot(): Snapshot {
  const p = this.player;
  return {
    joints: [...this.state.joints],
    gap: this.state.gap,
    tcp: forwardKinematics(ARM, this.state.joints.map((a) => a * DEG)),
    status: p.status,
    steps: p.steps,
    stepIndex: p.index,
    stepProgress: p.progress,
    cycle: p.cycle,
    speed: p.speed,
    loop: p.loop,
    error: p.error,
  };
}
```

`interrupt` pauses the program if it's running, and sets the manual gripper target to the current gap so the fingers stay put.

`publish` builds a fresh snapshot, resets the throttle timer and calls every listener. Calling a listener is how the store tells React “ask me again”; React then calls `getSnapshot`, sees a new object and re-renders the subscribed components.

`makeSnapshot` copies what the panel needs. `[...this.state.joints]` makes a new array, so later frames mutating `state` can't change a snapshot React is holding. The TCP is computed with forward kinematics here, so the panel doesn't need Three.js to show it.

GOING DEEPER

The alternatives, and why not

- **React context** re-renders every consumer on every change, which is the problem this design avoids.
- **Zustand** is this exact pattern as a library: a store outside React, `useSyncExternalStore` underneath, and transient subscriptions for per-frame reads.
- **Signals** (Preact, Solid, Angular) track dependencies per value and update only what read it.
- **Redux** works but adds actions and reducers for what is essentially one object.

The hand-written store is about 40 lines with no dependency, and shows exactly what the libraries do internally.

PART 2.5 · `src/ui/useSimulation.ts`

The hook

One line connects React to the store.

useSyncExternalStore

src/ui/useSimulation.ts · lines 1–10

```
import { useSyncExternalStore } from "react";
import type { Simulation, Snapshot } from "../sim/Simulation";

/**
 * Subscribes a component to the simulation. React re-renders it only when the
 * simulation publishes a new snapshot, not on every animation frame.
 */
export function useSimulation(sim: Simulation): Snapshot {
  return useSyncExternalStore(sim.subscribe, sim.getSnapshot);
}
```

Any component that calls `useSimulation(sim)` gets the current snapshot, and re-renders when the simulation publishes a new one, and only then.

`useSyncExternalStore` is React's built-in way to read data that lives outside React (browser APIs, or stores like Redux and Zustand). Compared with copying the data into `useState`, it can't show a stale or torn value during rendering, and there's no subscription code to write in each component.

PART 2.6 · `src/ui/Viewport.tsx`

Mounting the 3D view

The only place React touches Three.js.

A ref and an effect

src/ui/Viewport.tsx · lines 1–19

```
import { useEffect, useRef } from "react";
import { Viewer } from "../scene/Viewer";
import type { Simulation } from "../sim/Simulation";

/**
 * React's only contact with the 3D scene: it gives the Viewer a DOM element to
 * draw into, and disposes it on unmount. This component never re-renders for
 * simulation updates; the Viewer runs its own frame loop.
 */
export function Viewport({ sim }: { sim: Simulation }) {
  const containerRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    const viewer = new Viewer(containerRef.current!, sim);
    return () => viewer.dispose();
  }, [sim]);

  return <div ref={containerRef} className="viewport" aria-label="3D view of the robot work
cell" />;
}
```

`useRef` gives a handle to the real `<div>` once React has put it on the page. `containerRef.current` is that element (the `!` says it can't be null by the time the effect runs).

`useEffect` runs after the component appears on screen. It creates the `Viewer`, which builds the whole 3D scene inside the div. The function it returns is the **cleanup**, which React calls when the component goes away.

The dependency list `[sim]` means “only redo this if `sim` changes”, which it never does. So the viewer is created once, and this component never re-renders for simulation updates: it doesn't call `useSimulation`.

> In development, React's StrictMode deliberately mounts, unmounts and remounts every component once, to catch effects that don't clean up. That's why the viewer's `dispose()` has to undo everything; see stage.ts below.

PART 2.7 · `src/scene/Viewer.ts`

The 3D side

Owns the Three.js scene and the frame loop, and is the only thing that advances the simulation.

Fields

src/scene/Viewer.ts · lines 14–23

```
export class Viewer {
  private readonly stage: ReturnType<typeof createStage>;
  private readonly sim: Simulation;
  private readonly arm = new RobotArm(ARM, JOINTS);
  private readonly workcell: Workcell;
  private readonly tcp = new THREE.Vector3();
  private gripper: Gripper | null = null;
  private seenReset: number;
  private lastTime = performance.now();
  private disposed = false;
```

`ReturnType<typeof createStage>` means “whatever type `createStage` returns”, so the type doesn't have to be written out twice.

The arm is the same `RobotArm` class as Snack 1, reused unchanged. `tcp` is one `Vector3` reused every frame instead of allocating a new one. `seenReset` is the last `resetVersion` this viewer has handled.

Constructor

src/scene/Viewer.ts · lines 25–44

```
constructor(container: HTMLElement, sim: Simulation) {
  this.sim = sim;
  this.stage = createStage(container);
  this.stage.scene.add(this.arm.root);
  this.workcell = new Workcell(this.stage.scene);
  this.seenReset = sim.resetVersion;

  Gripper.load(`${import.meta.env.BASE_URL}models/gripper.glb`)
    .catch((err: unknown) => {
      console.error("Using a placeholder gripper:", err);
      return Gripper.placeholder();
    })
    .then((gripper) => {
      if (this.disposed) return; // unmounted while the model was loading
      this.gripper = gripper;
      this.arm.toolMount.add(gripper.object);
    });

  this.stage.renderer.setAnimationLoop(this.frame);
}
```

It builds the stage (renderer, camera, lights), adds the arm and the work cell, and starts loading the gripper model.

Loading is asynchronous, so it might finish **after** the viewer was disposed (for example, during StrictMode's remount). The `if (this.disposed) return;` check stops a dead viewer from attaching a gripper to a scene nobody is drawing.

`setAnimationLoop(this.frame)` asks Three.js to call `frame` before every screen refresh.

The frame

src/scene/Viewer.ts · lines 46–65

```
private frame = (time: number) => {
  const dt = Math.min((time - this.lastTime) / 1000, 0.05);
  this.lastTime = time;

  const { sim, arm } = this;
  sim.tick(dt);

  if (sim.resetVersion !== this.seenReset) {
    this.seenReset = sim.resetVersion;
    this.workcell.reset();
  }

  sim.state.joints.forEach((angle, i) => arm.setAngle(i, angle));
  arm.tcp.getWorldPosition(this.tcp);
  const gap = this.workcell.update(dt, this.tcp, sim.state.gap, this.gripper?.object ?? null);
  this.gripper?.setGap(gap);

  this.stage.controls.update();
  this.stage.renderer.render(this.stage.scene, this.stage.camera);
};
```

Every frame, in order:

1. Work out `dt`, capped at 50 ms so a tab returning from the background doesn't make the arm leap.
2. `sim.tick(dt)` advances the program (and decides whether to tell React).
3. If Reset was pressed since the last frame, put the part back.
4. Copy the four joint angles from `sim.state` onto the Three.js joints. This is the hand-off from plain data to the scene graph.
5. Read the tool's world position and let the work cell decide whether the part is grabbed, released or falling; it returns the finger gap to draw.
6. Draw.

React isn't anywhere in this list. The scene runs at the display's frame rate however busy React is.

`frame` is an arrow-function property so that `this` still means the viewer when Three.js calls it.

Dispose

src/scene/Viewer.ts · lines 67–72

```
  dispose(): void {
    this.disposed = true;
    this.stage.renderer.setAnimationLoop(null);
    this.stage.dispose();
  }
}
```

Stops the loop, then tears down the stage. Setting `disposed` first protects the pending gripper load.

PART 2.8 · `src/scene/Workcell.ts`

Picking up the part

The platform, the floor marker and the part, plus the rules for grabbing, releasing and dropping.

Constants and fields

src/scene/Workcell.ts · lines 4–17

```
const GRAVITY = 9.81;
const GRAB_DISTANCE = 0.02; // the part's center must be this close to the TCP to be grabbed

/**
 * Everything the robot works on: the platform, a marker for the floor spot, and
 * the part. Handles picking up (re-parenting the part to the gripper), letting
 * go, and the part dropping onto whatever is under it.
 */
export class Workcell {
  readonly part: THREE.Mesh;
  private readonly scene: THREE.Scene;
  private holder: THREE.Object3D | null = null;
  private fallSpeed = 0;
  private readonly partWorld = new THREE.Vector3();
```

`GRAB_DISTANCE`: the part counts as “between the fingers” when its center is within 2 cm of the tool center point. `holder` is the object holding the part (the gripper) or `null`. `fallSpeed` is for gravity. `partWorld` is another reused vector.

Building the scene objects

src/scene/Workcell.ts · lines 19–48

```
constructor(scene: THREE.Scene) {
  this.scene = scene;

  const platform = new THREE.Mesh(
    new THREE.BoxGeometry(PLATFORM.size, PLATFORM.height, PLATFORM.size),
    new THREE.MeshStandardMaterial({ color: 0x8a93a3, metalness: 0.3, roughness: 0.6 }),
  );
  platform.position.set(PLATFORM.x, PLATFORM.height / 2, PLATFORM.z);
  platform.castShadow = true;
  platform.receiveShadow = true;
  scene.add(platform);

  const marker = new THREE.Mesh(
    new THREE.PlaneGeometry(0.09, 0.09).rotateX(-Math.PI / 2),
    new THREE.MeshBasicMaterial({ color: 0x3b82f6, transparent: true, opacity: 0.35 }),
  );
  marker.position.set(FLOOR_SPOT.x, 0.002, FLOOR_SPOT.z);
  marker.rotation.y = yawAt(FLOOR_SPOT.x, FLOOR_SPOT.z);
  scene.add(marker);

  this.part = new THREE.Mesh(
    new THREE.BoxGeometry(PART_SIZE, PART_SIZE, PART_SIZE),
    new THREE.MeshStandardMaterial({ color: 0x14b8a6, metalness: 0.1, roughness: 0.5 }),
  );
  this.part.name = "Part";
  this.part.castShadow = true;
  this.part.receiveShadow = true;
  scene.add(this.part);
  this.reset();
}
```

Three meshes: the platform box (positioned up by half its height, since a box's origin is its center), a semi-transparent square marking the floor spot, and the 5 cm teal part. All cast and receive shadows except the marker.

`.rotateX(-Math.PI / 2)` on the plane geometry lays it flat; planes are created standing up, facing +Z.

Reset

src/scene/Workcell.ts · lines 50–57

```
/** Puts the part back on the floor spot, turned to line up with the gripper's fingers. */
reset(): void {
  if (this.holder) this.scene.attach(this.part);
  this.holder = null;
  this.fallSpeed = 0;
  this.part.position.set(FLOOR_SPOT.x, FLOOR_SPOT.y, FLOOR_SPOT.z);
  this.part.rotation.set(0, yawAt(FLOOR_SPOT.x, FLOOR_SPOT.z), 0);
}
```

If the part is being held, `scene.attach` takes it back from the gripper first. Then it's placed on the floor spot, rotated to the same yaw the base will use to reach it, so its faces line up with the fingers.

Grab and release

src/scene/Workcell.ts · lines 59–80

```
/**
 * Called every frame. Grabs or releases the part based on the commanded finger
 * gap, and returns the gap to draw: never narrower than the part while it's
 * between the fingers, so the fingers stop at the part instead of passing through it.
 */
update(dt: number, tcp: THREE.Vector3, commandedGap: number, gripper: THREE.Object3D | null):
number {
  this.part.getWorldPosition(this.partWorld);
  const between = this.holder !== null || this.partWorld.distanceTo(tcp) < GRAB_DISTANCE;

  if (!this.holder && gripper && between && commandedGap <= PART_SIZE + 0.001) {
    // attach() re-parents while keeping the world transform (Unity: SetParent(parent, true)).
    gripper.attach(this.part);
    this.holder = gripper;
    this.fallSpeed = 0;
  } else if (this.holder && commandedGap > PART_SIZE + 0.002) {
    this.scene.attach(this.part);
    this.holder = null;
  }

  if (!this.holder) this.settle(dt);
  return between ? Math.max(commandedGap, PART_SIZE) : commandedGap;
}
```

Called every frame with the tool position and the **commanded** gap from the simulation.

- The part is **between** the fingers if it's already held, or its center is close enough to the TCP.
- **Grab** when it's between the fingers, not yet held, and the commanded gap has closed to the part's width.
- **Release** when it's held and the gap opens wider than the part. The small tolerances (1 mm, 2 mm) keep it from grabbing and releasing on alternate frames.

Grabbing is **re-parenting**. `gripper.attach(part)` makes the part a child of the gripper while keeping its position and rotation in the world exactly where they were, so from then on it moves with the hand. `scene.attach(part)` does the reverse on release. In Unity terms, this is `SetParent(parent, worldPositionStays: true)`; plain `add()` would keep the local offset instead and make the part jump.

The return value is the gap to **draw**: if the part is between the fingers, never less than its width. The simulation can command a fully closed gripper, but the fingers visibly stop on the part instead of passing through it.

Gravity

src/scene/Workcell.ts · lines 82–98

```
  /** Falls under gravity until resting on the platform or the floor. */
  private settle(dt: number): void {
    const p = this.part.position; // the part is a direct child of the scene here, so this is
world space
    const onPlatform =
      Math.abs(p.x - PLATFORM.x) < PLATFORM.size / 2 && Math.abs(p.z - PLATFORM.z) <
PLATFORM.size / 2;
    const restY = (onPlatform ? PLATFORM.height : 0) + PART_SIZE / 2;

    if (p.y > restY) {
      this.fallSpeed += GRAVITY * dt;
      p.y = Math.max(restY, p.y - this.fallSpeed * dt);
    }
    if (p.y <= restY) {
      p.y = restY;
      this.fallSpeed = 0;
    }
  }
}
```

When nothing holds the part, it falls until it rests on something. `restY` is where its center sits when resting: on the platform if it's over the platform's footprint, otherwise on the floor.

Each frame, falling speed increases by g·dt (9.81 m/s² × the frame time) and the part moves down by speed·dt, without passing the resting height. This simple step-by-step integration is called Euler integration, and it's plenty for a 2 mm drop.

Lining up with the fingers

src/scene/Workcell.ts · lines 100–103

```
/** The base yaw the arm uses to reach (x, z): lines a part up with the fingers. */
function yawAt(x: number, z: number): number {
  return Math.atan2(z, -x);
}
```

The same formula as the base yaw in ik.ts. Turning the part to this angle means its faces are square to the fingers when the arm arrives.

GOING DEEPER

What a real cell would do

A real gripper reports whether it's holding something, through position feedback (fingers stopped before closing fully) or motor current, often with a part-presence sensor as well. In simulation, the honest version is a physics engine with friction between pads and part, like Snack 3. Re-parenting is the **kinematic attach** that many digital-twin and offline-programming tools use, because it's fast and never drops the part by accident.

PART 2.9 · `src/scene/stage.ts`

What changed in the stage

Mostly the same lights, camera and floor as Snack 1. Three changes make it work inside a React app.

It owns its canvas

src/scene/stage.ts · lines 9–16

```
export function createStage(container: HTMLElement) {
  // The stage owns its canvas, so a remount (React StrictMode does one in development)
  // always starts from a fresh WebGL context.
  const canvas = document.createElement("canvas");
  container.append(canvas);

  const renderer = new THREE.WebGLRenderer({ canvas, antialias: true });
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
```

In Snack 1 the canvas was in index.html. Here the stage creates a fresh `<canvas>` inside the container and removes it on dispose. After StrictMode's remount, the new viewer gets a brand-new canvas and WebGL context, with no leftovers from the first one.

It sizes itself to its container

src/scene/stage.ts · lines 57–67

```
const resize = () => {
  const width = container.clientWidth;
  const height = container.clientHeight;
  if (width === 0 || height === 0) return;
  renderer.setSize(width, height, false);
  camera.aspect = width / height;
  camera.updateProjectionMatrix();
};
const observer = new ResizeObserver(resize);
observer.observe(container);
resize();
```

Snack 1 listened to the window's `resize` event because the canvas filled the window. Here the 3D view shares the page with the panel, so a `ResizeObserver` watches the container itself: it fires when the container changes size for any reason, such as the window resizing or the layout switching to the stacked mobile view.

The `false` in `setSize` tells Three.js not to set the canvas's CSS size; the stylesheet already stretches it to fill the container.

It cleans up after itself

src/scene/stage.ts · lines 69–85

```
  function dispose() {
    observer.disconnect();
    controls.dispose();
    scene.traverse((object) => {
      if (object instanceof THREE.Mesh || object instanceof THREE.LineSegments) {
        object.geometry.dispose();
        const materials = Array.isArray(object.material) ? object.material : [object.material];
        materials.forEach((m: THREE.Material) => m.dispose());
      }
    });
    renderer.dispose();
    renderer.forceContextLoss(); // free the GPU context now rather than waiting for GC
    canvas.remove();
  }

  return { renderer, scene, camera, controls, dispose };
}
```

Undoes everything the stage created: stops watching size, removes the orbit controls' event listeners, frees every geometry and material on the GPU (meshes and line objects such as the grid), releases the renderer and its WebGL context, and removes the canvas. Without this, every remount would leave a running GPU context behind.

PART 2.10 · `src/arm/Gripper.ts`

What changed in the gripper

Same glTF model and loader as Snack 1; the animation moved out of the gripper.

setGap

src/arm/Gripper.ts · lines 4–5

```
/** Distance from a finger's origin to the inner face of its grip pad (see Snack 1's build-
gripper.ts). */
const PAD_INSET = 0.022;
```

src/arm/Gripper.ts · lines 59–64

```
/** Sets the distance between the grip pads, in meters. */
setGap(gap: number): void {
  const x = gap / 2 + PAD_INSET;
  this.left.position.x = -x;
  this.right.position.x = x;
}
```

In Snack 1 the gripper animated itself between open and closed. Now the simulation owns the gap, as part of the robot's state, and the viewer passes the gap to draw (from the work cell) to `setGap` each frame.

Each finger's origin is its root, and its grip pad's inner face is 2.2 cm in from there (from the geometry in Snack 1's build-gripper.ts). So for a gap `g` between the pads, each finger sits at `g/2 + 0.022` from the center.

PART 2.11 · `src/ui/ControlPanel.tsx`

The panel

Plain React components. They read the snapshot and call simulation commands; they never touch Three.js.

Setup and the render counter

src/ui/ControlPanel.tsx · lines 7–17

```
const SPEEDS = [0.5, 1, 2, 4];
const KIND_LABEL: Record<Step["kind"], string> = { moveJ: "MoveJ", moveL: "MoveL", gripper:
"Grip", home: "MoveJ" };

/** Counts committed renders of the panel, outside React state so counting doesn't cause
renders. */
let panelCommits = 0;

export function ControlPanel({ sim }: { sim: Simulation }) {
  const snap = useSimulation(sim);
  useEffect(() => {
    panelCommits++;
  });
```

`KIND_LABEL` maps each step kind to the badge shown in the step list. `Record<Step["kind"], string>` means “an object with one string for every possible `kind`”, so adding a new step kind without a label is a compile error.

`panelCommits` counts how many times the panel has actually rendered. It's a plain module variable on purpose: storing it in React state would itself cause a render each time it changed. A `useEffect` with no dependency list runs after every render, so it increments once per render.

The panel component

src/ui/ControlPanel.tsx · lines 19–32

```
  return (
    <aside className="panel">
      <header>
        <h1>Robot Control Panel</h1>
        <p className="sub">React · Three.js · TypeScript</p>
      </header>
      <ProgramSection sim={sim} snap={snap} />
      <JointsSection sim={sim} snap={snap} />
      <GripperSection sim={sim} />
      <TcpSection snap={snap} />
      <Stats sim={sim} />
    </aside>
  );
}
```

`useSimulation(sim)` subscribes the panel to the store. The panel passes the snapshot and the simulation down to small section components. When a new snapshot arrives, the panel re-renders, and the 3D view isn't involved.

Program controls

src/ui/ControlPanel.tsx · lines 34–64

```
function ProgramSection({ sim, snap }: { sim: Simulation; snap: Snapshot }) {
  const playing = snap.status === "playing";
  const active = playing || snap.status === "paused";

  return (
    <section>
      <h2>Program: pick and place</h2>
      <div className="transport">
        <button type="button" className="primary" onClick={() => (playing ? sim.pause() :
```

```
sim.play())}>
          {playing ? "Pause" : snap.status === "paused" ? "Resume" : snap.status === "done" ?
"Run next cycle" : "Play"}
        </button>
        <button type="button" onClick={() => sim.reset()}>
          Reset
        </button>
      </div>

      <div className="options">
        <label>
          Speed{" "}
          <select value={String(snap.speed)} onChange={(e) =>
sim.setSpeed(Number(e.currentTarget.value))}>
            {SPEEDS.map((s) => (
              <option key={s} value={String(s)}>
                {s}×
              </option>
            ))}
          </select>
        </label>
        <label>
          <input type="checkbox" checked={snap.loop} onChange={(e) =>
sim.setLoop(e.currentTarget.checked)} /> Loop
        </label>
      </div>
```

The main button's label and action depend on the status: Pause while playing, Resume when paused, “Run next cycle” after a cycle finished with loop off, otherwise Play.

The speed `<select>` and loop checkbox are **controlled inputs**: their displayed value always comes from the snapshot, and changes go through the simulation. The `<select>` works with strings, hence `String()` and `Number()`. `{" "}` is how JSX adds a space between “Speed” and the dropdown.

Status line and step list

src/ui/ControlPanel.tsx · lines 66–85

```
      <p className={snap.status === "error" ? "status error" : "status"} role="status">
        {statusText(snap)}
      </p>

      <ol className="steps">
        {snap.steps.map((step, i) => {
          const current = active && i === snap.stepIndex;
          const done = active && i < snap.stepIndex;
          return (
            <li key={i} className={current ? "current" : done ? "done" : undefined}>
              <span className="label">{step.label}</span>
              <span className="kind">{KIND_LABEL[step.kind]}</span>
              {current && <span className="progress" style={{ width: `${snap.stepProgress *
100}%` }} />}
            </li>
          );
        })}
      </ol>
    </section>
  );
}
```

`role="status"` tells screen readers to announce changes to this line.

The step list highlights the current step and ticks off finished ones, but only while a run is active, so the idle list looks clean. The current step gets a thin progress bar whose width is the step's progress as a percentage.

`key={i}` helps React match list items between renders. Using the index is fine here because the list is never reordered.

Status text

src/ui/ControlPanel.tsx · lines 87–101

```
function statusText(snap: Snapshot): string {
  const where = `Cycle ${snap.cycle + 1}, step ${snap.stepIndex + 1} of ${snap.steps.length}`;
  switch (snap.status) {
    case "idle":
      return "Ready. Press Play to run the program.";
    case "playing":
      return where;
    case "paused":
      return `Paused at ${where.toLowerCase()}. Moving a joint pauses the program.`;
    case "done":
      return `Cycle ${snap.cycle} finished.`;
    case "error":
      return snap.error ?? "Error";
  }
}
```

A `switch` covering every status. Because `status` is a union type, TypeScript can tell all five cases are handled, so the function needs no fallback `return`.

Joint sliders

src/ui/ControlPanel.tsx · lines 103–123

```
function JointsSection({ sim, snap }: { sim: Simulation; snap: Snapshot }) {
  return (
    <section>
      <h2>Joints</h2>
      {JOINTS.map((joint, i) => (
        <label key={joint.name} className="slider">
          <span className="name">{joint.name}</span>
          <output>{snap.joints[i].toFixed(1)}°</output>
          <input
            type="range"
            min={joint.min}
            max={joint.max}
            step="any"
            value={snap.joints[i]}
            onChange={(e) => sim.setJoint(i, e.currentTarget.valueAsNumber)}
          />
        </label>
      ))}
    </section>
  );
}
```

One slider per joint, built from the same `JOINTS` list the arm is built from. The value comes from the snapshot, so during playback the sliders move by themselves (updated up to 15 times a second).

Dragging one calls `sim.setJoint`, which pauses the program and moves the joint. `valueAsNumber` reads the slider as a number directly. `step="any"` allows any value, so the program's fractional angles aren't rounded to a slider step.

Gripper buttons and TCP readout

src/ui/ControlPanel.tsx · lines 125–154

```
function GripperSection({ sim }: { sim: Simulation }) {
  return (
    <section>
      <h2>Gripper</h2>
      <div className="transport">
        <button type="button" onClick={() => sim.setGripper(true)}>
          Open
```

```
        </button>
        <button type="button" onClick={() => sim.setGripper(false)}>
          Close
        </button>
      </div>
    </section>
  );
}

function TcpSection({ snap }: { snap: Snapshot }) {
  const { x, y, z } = snap.tcp;
  return (
    <section>
      <h2>Tool center point (m)</h2>
      <p className="tcp">
        <span>x {x.toFixed(3)}</span>
        <span>y {y.toFixed(3)}</span>
        <span>z {z.toFixed(3)}</span>
        <span>gap {(snap.gap * 1000).toFixed(0)} mm</span>
      </p>
    </section>
  );
}
```

Open and Close call `setGripper`. The readout shows the tool position (from the snapshot's forward kinematics) and the gap in millimeters.

Stats without renders

src/ui/ControlPanel.tsx · lines 156–172

```
/**
 * Frames rendered vs. times React rendered the panel. Updated by writing to the
 * DOM directly twice a second, so showing the numbers doesn't add renders.
 */
function Stats({ sim }: { sim: Simulation }) {
  const ref = useRef<HTMLParagraphElement>(null);
  useEffect(() => {
    const id = window.setInterval(() => {
      if (ref.current) {
        ref.current.textContent =
          `3D frames: ${sim.frames.toLocaleString()} · panel renders:
${panelCommits.toLocaleString()}`;
      }
    }, 500);
    return () => window.clearInterval(id);
  }, [sim]);
  return <p ref={ref} className="stats" />;
}
```

This line proves the design works: it shows how many 3D frames have been drawn next to how many times the panel has rendered.

Showing those numbers through React state would cause renders of its own, ruining the measurement. Instead, a timer writes them straight into the paragraph's text twice a second through a ref. The cleanup function clears the timer when the component goes away.

PART 2.12 · `src/App.tsx`

Putting it together

One simulation, two consumers

src/App.tsx · lines 1–16

```
import { useState } from "react";
import { Simulation } from "./sim/Simulation";
import { ControlPanel } from "./ui/ControlPanel";
import { Viewport } from "./ui/Viewport";

export default function App() {
  // One simulation for the app's lifetime, shared by the panel and the 3D view.
  const [sim] = useState(() => new Simulation());

  return (
    <div className="app">
      <ControlPanel sim={sim} />
      <Viewport sim={sim} />
    </div>
  );
}
```

`useState(() => new Simulation())` creates the simulation once and keeps it for the app's lifetime. Passing a function (rather than `useState(new Simulation())`) means it's only constructed on the first render, not thrown away on every render.

Both the panel and the viewport receive the same `sim`. They never talk to each other directly; each only talks to the simulation.

CHAPTER 2

Interview questions

Q1 **Why not keep the simulation's state in `useState` and update it every frame?**

Every update would re-render the panel at 60–144 Hz on the same thread as the render loop, for numbers a person reads a few times a second. Keeping the state in a plain object lets the 3D loop mutate and read it directly, and React gets an immutable snapshot only when a discrete event happens or at most 15 times a second.

Q2 **What does `getSnapshot` have to guarantee?**

That it returns the same object, by `Object.is`, as long as the data hasn't changed, and a new object when it has. Returning a fresh object each call looks like a change every time, so React re-renders in a loop and errors. Returning the same object after mutating it hides changes. Hence: build a new frozen snapshot on publish, and return the cached one in between.

Q3 **What is tearing?**

With concurrent rendering, a render can be paused and resumed. If components read mutable external data during render, some can see the old value and some the new one in the same commit, so the UI is inconsistent. `useSyncExternalStore` re-checks the snapshot before committing and re-renders synchronously if it changed.

Q4 **Walk me through your IK. What happens when the target can't be reached?**

Base yaw from atan2 of the target's horizontal position. With the tool fixed vertical, the wrist joint must be directly above the target, so solve the two-link arm to the wrist joint: law of cosines for the elbow, then the shoulder as the direction to the wrist minus the angle the bent elbow adds. The wrist takes whatever makes the pitch angles sum to 180°. If the cosine falls outside [−1, 1] there is no triangle and it returns null; a separate check rejects solutions outside the joint limits.

Q5 **moveJ or moveL: when do you use each, and what can go wrong with moveL?**

moveJ for big moves in open space: fastest, always feasible between two valid poses, but the tool's path curves. moveL near parts, for approach and retreat, where the path matters. moveL can hit an unreachable point or a singularity halfway, so the path should be validated before running it.

Q6 **Your component works in production but loads twice and leaks in development. Why?**

StrictMode mounts, unmounts and remounts components in development to expose effects without proper cleanup. The fix isn't to disable StrictMode; it's a cleanup that stops the loop, disposes resources and the renderer, and a guard so async work that finishes after unmount doesn't touch a dead scene.

Q7 **What's the difference between `add` and `attach` in Three.js?**

`add` keeps the child's local transform, so the object jumps if the new parent is somewhere else. `attach` keeps the world transform by setting local = inverse(parent world) × child world. It's Unity's `SetParent(parent, worldPositionStays: true)`.

Q8 **How do you test a simulation that normally runs in a browser?**

Keep the simulation pure TypeScript with time passed in. Then tests run in Node: FK(IK(p)) = p across the work area, unreachable points return null, a full cycle ends at home, the gripper closes exactly on the part, nothing touches the floor over two loops, pause and resume, and speed scaling.

Q9 **How would you let users edit the program?**

The program is already data: a discriminated union of steps. An editor would edit that array immutably (cheap undo and redo), validate each step on change by running IK along it, and show unreachable steps in the panel before anything runs. Serialization is just JSON.

CHAPTER 2

Exercises

2.1 Elbow down

Add an option for the “elbow down” solution and a test that FK of both solutions reaches the same point.

Hint. Negate the elbow angle; the shoulder formula changes sign in one place. Expect different joint-limit failures for each branch.

2.2 Validate before moving

Before running, check every moveL by sampling 20 points along its line through IK and the limits, and show any failure in the panel.

Hint. This belongs in the simulation, not the component. The panel just renders a list of problems from the snapshot.

2.3 A third station

Add a third platform and a program that cycles the part through all three.

Hint. `pickAndPlace(from, to)` already builds a cycle from two stations; the loop logic needs a modulo 3 instead of even/odd.

2.4 Smarter publishing

Replace the fixed 15 Hz throttle with “publish when a displayed value changes at display precision” (0.1°, 1 mm). Count renders with the React Profiler.

Hint. Compare against the last published snapshot, rounded the way the panel rounds.

2.5 Real velocity profiles

Replace smoothstep with a trapezoidal profile per joint, given a maximum speed and acceleration, and size each moveJ by its slowest joint.

Hint. Duration = the largest of each joint's time to travel its distance. Every joint then follows the same normalized profile.

APPENDIX A

Running the projects

Every project needs Node.js 22 or newer; Snacks 1 and 5 also need Docker Desktop.

All five at once: from `Portfolio/vention/`, `npm start` starts Snack 5 in Docker (http://localhost:8081), prints Snack 4's quote and runs its tests, and serves the arm, control panel and conveyor on :5173, :5174 and :5175. `npm run stop` stops the database; `start:open` also opens the pages; `start:mongo` uses MongoDB. Or one at a time, from each project's folder:

Project	Commands
`snack1-robot-arm`	`npm ci` · `npm start` (database + API in Docker, then the arm at http://localhost:5173; `npm run dev` for the arm alone) · `npm test` · `npm run check` (type-check + tests) · `npm run build:gripper` regenerates the .glb · needs Node 22 (Appendix C) · pick and place: Appendix D
`snack2-control-panel`	`npm ci` · `npm run dev` · `npm test` (IK and program tests) · `npm run build`
`snack3-conveyor-physics`	`npm ci` · `npm run dev` · `npm test` · `npm run bench` (headless physics benchmark)
`snack4-refactor-kata`	`npm ci` · `npm test` (spec + golden master) · `npm run typecheck` · `npm run demo`
`snack5-inventory`	`docker compose --profile postgres up --build` (or `--profile mongo`), then http://localhost:8081 · `docker compose --profile test run --rm --build api-test` runs the contract tests on both databases · without Docker: `cd api; npm install; npm test`

Snack 4's full commit history ships as `history/refactor-steps.bundle`: `git clone history/refactor-steps.bundle steps`, then `git log --stat` in the new folder.

APPENDIX B

Glossary

AABB
Axis-aligned bounding box: a box whose faces line up with the world axes, described by its center and half sizes or its min and max corners.

Accessor (glTF)
Says how to read a range of a bufferView as numbers: component type, element type (VEC3, SCALAR…), count, bounds.

ACID
Atomicity, consistency, isolation, durability: the guarantees a database transaction gives.

Bin packing
Packing items into the fewest fixed-size bins; cut lists into stock bars are an instance.

Broad phase
The cheap first pass of collision detection that finds pairs which might touch.

Buffer view (glTF)
A byte range within a buffer, usable directly as a GPU buffer.

Characterization test
A test that records what code currently does, to detect changes, rather than what it should do.

CHECK constraint
A rule on a table that the database enforces on every write, rejecting rows that break it.

Conditional update
An update whose filter contains the rule (“only if enough is available”), so check and write are one atomic step.

Connection point
A named spot on a kind of object, stored as fractions of its size, where it can be joined to another.

Contract test
One test suite run against every implementation of an interface, so they behave the same.

CORS
Cross-origin resource sharing: headers by which a server lets pages from other origins read its responses in a browser.

Counting sort
A non-comparison sort that buckets items by a small integer key in O(n).

CTE
Common table expression (WITH … AS): names a sub-query; in Postgres it can also chain data-changing statements.

Deadlock
Two transactions each waiting for a lock the other holds; avoided by locking in a consistent order.

Deadlock
A state where each party waits for something only another waiting party can provide, so none can proceed.

Denavit–Hartenberg
A standard way to describe each joint of a robot arm with four parameters, giving one transform matrix per joint.

Diverter
A device on a conveyor that steers items off the line to one side, here a hinged flap.

Document database
A database storing self-contained JSON-like documents, which can embed arrays and sub-objects (MongoDB).

Draw call
One request from the CPU to the GPU to draw geometry with a given state. Each has fixed overhead.

Emissive
Light a material gives off regardless of the scene's lights; used to make the selected housing glow.

Event delegation
One listener on a container handling events from its children, identified by attributes, so re-rendered children need no new listeners.

First-fit decreasing
A bin-packing heuristic: sort items largest first, put each in the first bin with room.

Fixed time step
Advancing a simulation in equal small steps, independent of the display's frame rate, so results don't depend on it.

Foreign key
A column that must match a key in another table, so references are always valid.

Forward kinematics
Computing the tool's position from joint angles. Always one answer.

Frustum culling
Skipping objects whose bounding sphere is outside the camera's view.

Golden master
Recorded outputs of existing code for many inputs, compared against new code.

Idempotency key
A client-chosen key that makes retrying a request safe: the server returns the stored result instead of acting twice.

Impulse
An instant change in momentum, the solver's unit of correction.

Instancing
Drawing one geometry many times in one draw call, with per-instance data such as a matrix and color.

Interpolation (α)
Drawing a moment between the last two physics states, by the fraction α of a step left over.

Inverse kinematics
Computing joint angles that put the tool at a target. Zero, several or infinitely many answers.

Island
A group of bodies connected by contacts; they sleep and wake together.

Isolation level
How much concurrent transactions can see of each other (READ COMMITTED, REPEATABLE READ, SERIALIZABLE).

Jacobian
The matrix of partial derivatives of tool position with respect to joint angles; maps joint speeds to tool speed.

jsonb
PostgreSQL's binary JSON type: stores a document in a column, with operators and indexes to query inside it.

Kanban
Pull-based production: a step makes or sends an item only when the next step signals it needs one.

Kerf
The width of material a saw blade removes with each cut.

Kinematic body
A body moved by code, with a velocity, that pushes others but is never pushed itself.

Kinematic simulation
Motion set directly (positions and velocities) rather than computed from forces.

Kustomize overlay
A directory that builds on a base set of Kubernetes manifests, adding or changing resources.

Little-endian
Byte order with the least significant byte first; required by the glTF container.

Local frame
A coordinate system attached to an object, in which its own geometry is simple; points are transformed into it.

Lost update
A race where two transactions read the same value and one's write silently overwrites the other's.

moveJ / moveL
Robot moves that interpolate joint angles (curved tool path) or the tool position (straight line).

Narrow phase
The exact collision test on candidate pairs, producing contact normal and depth.

Normal
A unit vector perpendicular to a surface; drives lighting and, in physics, the direction of contact forces.

Normalization
Storing each fact once in its own table, related by keys, instead of repeating it.

Object pool
Preallocated objects or slots reused instead of created and destroyed.

ON DELETE CASCADE
A foreign-key option that deletes the referencing rows when the referenced row is deleted.

Origin
The scheme, host and port of a page (http://localhost:5173); browsers isolate pages by origin.

PBR
Physically based rendering: materials described by base color, metalness and roughness, lit consistently in any environment.

Preflight
The OPTIONS request a browser sends before a cross-origin request that isn't simple, asking whether it's allowed.

Pure function
A function whose output depends only on its inputs and which has no side effects.

Quaternion
Four numbers representing a 3D rotation, free of gimbal lock and easy to interpolate.

Raycasting
Intersecting a ray (here from the camera through the mouse) with scene geometry, for picking.

Re-parenting
Moving an object to a new parent in the scene graph; `attach` keeps its world pose, `add` keeps its local one.

Readiness probe
A Kubernetes check that decides whether a pod receives traffic; separate from liveness, which decides restarts.

Reconciliation
React's comparison of a new render's output with the previous one, to find the DOM changes to make.

Refactoring
Changing the structure of code without changing its behaviour, in small steps.

Replica set
A group of MongoDB servers holding the same data; required for multi-document transactions.

Repository pattern
Hiding storage behind an interface of domain operations, so the rest of the code doesn't know the database.

Ring buffer
A fixed array used as a queue by wrapping an index, so nothing is shifted or allocated.

Row lock
A lock on specific rows (SELECT … FOR UPDATE) that makes other writers wait until the transaction ends.

Rule engine
A small interpreter that evaluates conditions stored as data, instead of decisions written as code.

Saga
A multi-step operation made of individually atomic steps, with compensating steps to undo earlier ones on failure.

Scene graph
A tree of objects whose transforms compose from parent to child.

Seam
A place where code's behaviour can be observed or substituted without editing it, which makes testing possible.

Semi-implicit Euler
Integration that updates velocity first, then position from the new velocity; stable and cheap.

Sequential impulses
Solving contact constraints one at a time and repeating, a form of projected Gauss–Seidel.

Shock propagation
Treating the lower body of a vertical contact as immovable on a final, bottom-up pass, so stacks settle.

Singularity
A pose where the arm loses the ability to move the tool in some direction; the Jacobian loses rank.

Slop
Overlap allowed between resting bodies before position correction acts, to prevent jitter.

Smoothstep
The easing curve $3t^2 - 2t^3$: from 0 to 1 with zero slope at both ends.

Spatial hash
A uniform grid of cells stored in a hash table, for finding nearby objects in O(n).

Spiral of death
A slow frame causing more physics steps, making the next frame slower; avoided by capping steps.

Stand-in object
A temporary value built only so existing logic (here, rule matching) can be applied to something that doesn't exist yet.

StatefulSet
A Kubernetes workload with stable pod names and a persistent volume per pod, used for databases.

Structure of arrays
Storing each property of many entities in its own array, instead of one object per entity.

TCP
Tool center point: the point on the tool that motion is planned for; here, between the fingertips.

Tearing (React)
Different components showing different versions of external data in the same commit.

Tessellation
Converting exact surfaces (from CAD) into triangles for rendering.

Upsert
Insert a row, or update it if one with the same key exists, in one statement (INSERT ... ON CONFLICT DO UPDATE).

Winding order
The order a triangle's corners are listed; counter-clockwise as seen marks the front face.